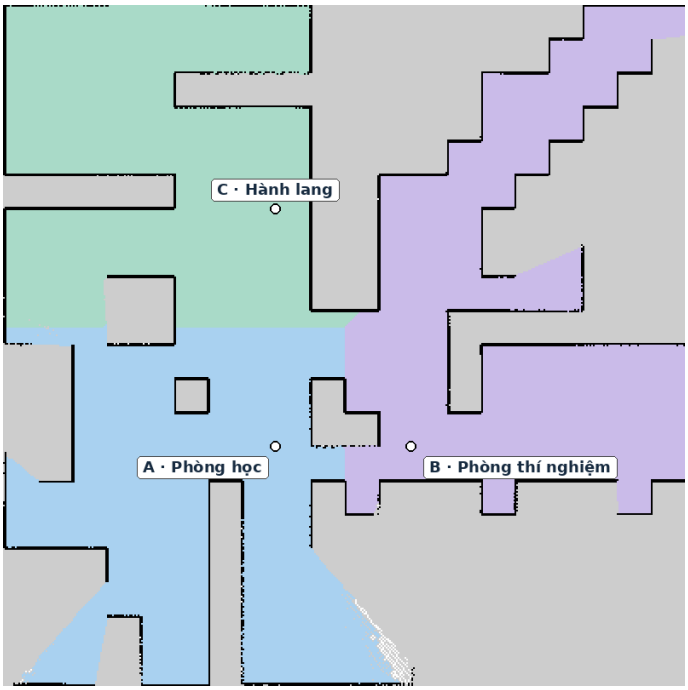
BÁO CÁO THỰC HÀNH

HỌC PHẦN TƯƠNG TÁC NGƯỜI – ROBOT

BÀI THỰC HÀNH 7

PHÂN VÙNG BẢN ĐỒ VÀ ĐIỀU HƯỚNG ROBOT BẰNG GIỌNG NÓI

Ubuntu 22.04 • ROS 2 Humble • TurtleBot 4 Simulation
Nav2 • Semantic Mapping • Vosk tiếng Việt



Hình 1. Ba vùng chức năng A, B và C trên bản đồ mê cung.

Sinh viên thực hiện: .....

Mã số sinh viên: .....

Lớp / Nhóm: .....

Giảng viên hướng dẫn: .....

Ngày lập báo cáo: 07/11/2026
Bản báo cáo chi tiết – bổ sung cho bản PDF tóm tắt 2 trang

TÓM TẮT BÁO CÁO VÀ PHẠM VI KẾT QUẢ

Bài thực hành xây dựng một hệ thống điều hướng có lớp ngữ nghĩa cho robot TurtleBot 4 trong môi trường mô phỏng. Thay vì chỉ định trực tiếp một cặp tọa độ trên RViz, người dùng có thể nói tên khu vực hoặc một chuỗi khu vực cần đi qua. Hệ thống phải chuyển giọng nói thành văn bản, hiểu đúng thứ tự tuyến, xác định các đích hợp lệ trên bản đồ và giao việc di chuyển cho Nav2.

Giải pháp gồm ba phần chính: phân vùng bản đồ Occupancy Grid theo hạt giống chức năng, nhận dạng và phân tích câu tiếng Việt, và một ROS 2 node quản lý action NavigateToPose tuần tự. Giao diện hiển thị bản đồ có nhãn, câu ASR đang xử lý, tuyến đã hiểu, vị trí TF của robot, đường kế hoạch và trạng thái thực thi. Các quyết định được ghi log để đối chiếu sau thử nghiệm.

Kết quả thực nghiệm được chia thành các tầng bằng chứng. Kiểm thử logic và action server giả xác nhận các quy tắc chương trình. Bộ WAV tiếng Việt tổng hợp kiểm tra phần âm thanh – ASR – parser. Thử nghiệm tích hợp sử dụng kết quả giải mã thực để điều hướng robot trong Gazebo với Nav2 thật. Không xem âm thanh tổng hợp là bản ghi giọng nói của sinh viên và không dùng hoạt cảnh giả để thay chuyển động robot.

Bảng 1. Tóm tắt các kết quả và trạng thái nộp bài.

Chỉ tiêu	Kết quả kiểm chứng
Số khu vực có nhãn	3: A – Phòng học; B – Phòng thí nghiệm; C – Hành lang
Kiểm thử tự động	19: 10 core + 3 decoder/UTF-8 + 6 luồng action giả
Điều hướng trong Gazebo	4 tuyến hợp lệ / 5 goal Nav2 thành công
Khu vực không tồn tại	D được nhận dạng là “đê”, parser từ chối, không gửi goal
Sai số vị trí lớn nhất	0.237 m; trung bình 0.224 m
Dữ liệu ground truth	440 mẫu chỉ đọc để đối chiếu; không dùng làm bộ điều khiển
Video demo	264,2 giây; hình Nav2/Gazebo thật; âm thanh đầu vào tổng hợp
GitHub public	Chưa xác nhận push public; source đã có repository local

Các tên phòng trong báo cáo là nhãn chức năng do người dùng gán trên world maze, không phải kết quả nhận diện kiến trúc phòng từ ảnh. Bản phân vùng hiện tại là bán tự động: con người định nghĩa hạt giống và tên, thuật toán tự lan nhãn trên vùng free. Những giới hạn này được giữ nhất quán trong mô tả, bảng kết quả và kết luận.

MỤC LỤC VÀ QUY ƯỚC ĐỌC BÁO CÁO

Mục lục dưới đây có số trang tham chiếu; có thể cập nhật trường trong Word bằng Ctrl+A → F9.

1. MỤC TIÊU, YÊU CẦU VÀ TIÊU CHÍ NGHIỆM THU 4
2. MÔI TRƯỜNG, CÔNG NGHỆ VÀ CẤU TRÚC SOURCE 5
3. KIẾN TRÚC TỔNG THỂ VÀ LUỒNG DỮ LIỆU 6
4. BẢN ĐỒ OCCUPANCY GRID VÀ BIẾN ĐỔI TỌA ĐỘ 7
5. PHƯƠNG PHÁP PHÂN VÙNG BÁN TỰ ĐỘNG 8
6. GẮN NHÃN, DỮ LIỆU VÙNG VÀ ĐÍCH HỢP LỆ 9
7. THU ÂM VÀ NHẬN DẠNG GIỌNG NÓI TIẾNG VIỆT 10
8. CHUẨN HÓA VĂN BẢN, ALIAS VÀ TUYẾN NHIỀU ĐÍCH 11
9. ĐIỀU HƯỚNG BẰNG NAV2 VÀ QUẢN LÝ ACTION 12
10. ĐỊNH VỊ, TRÁNH VẬT CẢN VÀ ĐIỀU KIỆN SẴN SÀNG 13
11. GIAO DIỆN, TOPIC ROS VÀ DỮ LIỆU HIỂN THỊ 14
12. QUY TRÌNH CÀI ĐẶT VÀ CHẠY THỬ 15
13. THIẾT KẾ THỬ NGHIỆM VÀ NGUỒN BẰNG CHỨNG 16
14. BẢNG KIỂM THỬ TỰ ĐỘNG CHI TIẾT 17
15. KẾT QUẢ ASR VÀ BÀI HỌC TỪ MẪU NHẬN SAI 18
16. KẾT QUẢ ĐIỀU HƯỚNG GAZEBO THEO TỪNG GOAL 19
17. PHÂN TÍCH SAI SỐ, QUỸ ĐẠO VÀ HAZARD 20
18. ĐỐI CHIẾU QUỸ ĐẠO VÀ XỬ LÝ KHU VỰC LẠ 21
19. LỖI ĐÃ GẶP, CÁCH KHẮC PHỤC VÀ KIỂM SOÁT CHẤT LƯỢNG 22
20. GIỚI HẠN, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN 23
21. HƯỚNG DẪN QUAY DEMO VÀ DANH MỤC NỘP BÀI 24
22. PHỤ LỤC: LỆNH TÁI HIỆN, TRÍCH LOG VÀ TÀI LIỆU 25

Quy ước và thuật ngữ

Thuật ngữ	Ý nghĩa sử dụng trong báo cáo
Occupancy Grid	Lưới ô mô tả vùng đã biết trống, vật cản và vùng chưa biết.
Semantic region	Vùng hình học gắn ID/tên, alias và một goal điều hướng.
ASR / STT	Chuyển tín hiệu tiếng nói thành văn bản.
Goal / route	Goal là một đích; route là chuỗi ID khu vực có thứ tự.
Clearance	Khoảng cách từ tâm ô/robot đến phần biên được kiểm tra.
Ground truth	Vị trí mô phỏng dùng chỉ đọc cho initial pose/đối chiếu theo phạm vi nêu rõ.
Mock / synthetic	Server giả phục vụ test / âm thanh được tổng hợp, không phải giọng người thật.

1. MỤC TIÊU, YÊU CẦU VÀ TIÊU CHÍ NGHIỆM THU

1.1. Bài toán cần giải quyết

Trong ứng dụng robot phục vụ ở trường học hoặc phòng thí nghiệm, người vận hành thường nhớ tên khu vực hơn là tọa độ x, y. Một giao diện chỉ có nút Nav2 Goal vẫn đòi hỏi người dùng hiểu bản đồ và trực tiếp chọn vị trí. Bài thực hành bổ sung lớp ngữ nghĩa để liên kết ngôn ngữ tự nhiên với các đích được khảo sát sẵn.

Đầu vào là bản đồ 2D đã tạo từ trước và một câu nói tiếng Việt. Đầu ra không chỉ là ID vùng được nhắc đến mà còn là một trình tự NavigateToPose có kiểm tra điều kiện. Khi câu nói yêu cầu “từ A đến B”, vùng nguồn A là một điểm phải đi qua; vị trí hiện tại khác A không được làm chương trình tự bỏ A và chạy thẳng đến B.

Bảng 2. Đối chiếu yêu cầu đề bài với thiết kế và bằng chứng.

Yêu cầu	Cách đáp ứng	Bằng chứng
Tối thiểu 3 vùng	Geodesic Voronoi với 3 hạt giống A/B/C	Ảnh segmented_map.png; lưới nhãn; regions.json
Đích hợp lệ	Known-free, đúng vùng, clearance ≥0,32 m	Kiểm thử map; metadata từng goal
Tối thiểu 3 câu lệnh	Alias tên phòng/ID; hỗ trợ nguồn–đích	Test parser; WAV → ASR; log tích hợp
Di chuyển và tránh vật cản	Nav2 + LiDAR + AMCL trên TurtleBot 4	Action status; TF cuối; telemetry Gazebo
Xử lý khu vực lạ	Từ chối toàn tuyến trước send_goal	Ca D/đê; test vùng Z trong tuyến
Source, video, báo cáo	Repository local, ZIP, MP4, PDF và Word	GitHub public còn chờ kết nối

1.2. Tiêu chí đánh giá được chọn

Một tuyến hợp lệ chỉ được coi là hoàn thành khi mọi goal xuất hiện đúng thứ tự, đều có status=4 và thỏa kiểm tra sai số vị trí sau action. Ca vùng lạ được coi là xử lý đúng nếu có thông báo từ chối và không xuất hiện GOAL_SENT. Bộ kiểm thử còn theo dõi các sự kiện hazard từ driver để phát hiện lỗi trong lượt chạy.

Thời gian thực hiện một tuần là thời hạn của đề bài, không được dùng để suy ra rằng đã đo liên tục một tuần. Các con số trong báo cáo là dữ liệu từ lượt kiểm thử ngày 07/11/2026 được lưu trong artifacts. Những mở rộng chưa triển khai được trình bày riêng, không tính như tiêu chí đã đạt.

2. MÔI TRƯỜNG, CÔNG NGHỆ VÀ CẤU TRÚC SOURCE

Hệ thống triển khai trên Ubuntu 22.04 với ROS 2 Humble và TurtleBot 4 Lite trong Ignition Gazebo Fortress. Nav2 thực hiện lập kế hoạch và điều khiển; AMCL ước lượng pose trên bản đồ tĩnh. Python được dùng cho xử lý bản đồ, nhận dạng lời nói, giao diện và action client. Venv có system-site-packages để dùng các binding rclpy của bản cài ROS apt.

Bảng 3. Các thành phần phần mềm chính.

Thành phần	Công nghệ / cấu hình	Vai trò
Robot mô phỏng	TurtleBot 4 Lite + Create 3	Chuyển động, odometry, dock, hazard và LiDAR
Điều hướng	Nav2; NavFn planner; DWB controller	Tạo đường và vận tốc tránh vật cản
Định vị	AMCL; chuỗi TF map/odom/base_link	Pose trong frame map
Bản đồ	PGM 401×401; 0,05 m/ô	Occupancy Grid gốc cho maze
Xử lý ảnh	OpenCV + NumPy + Pillow	Mask free, distance transform, nhãn và ảnh minh họa
Giọng nói	arecord; Vosk 0.3.45; small-vn-0.4	Thu PCM và nhận dạng tiếng Việt offline
Giao diện	Tkinter + Pillow	Bản đồ semantic, transcript, trạng thái và video
Kiểm chứng	unittest; mock action; log JSON/JSONL	Tách test logic và thử nghiệm simulator

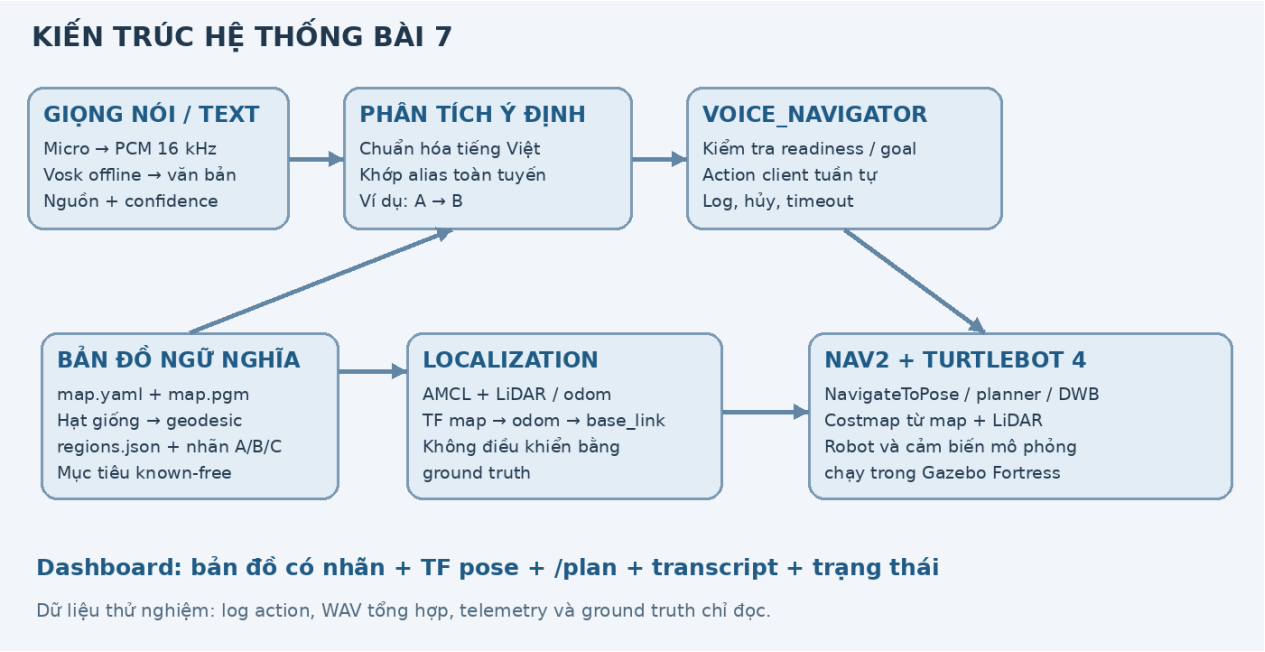
2.1. Hai package ROS được sử dụng

```
src/
  turtlebot4_slam_nav/    # launch world, AMCL, Nav2, config, bản đồ`
  turtlebot4_voice_nav/
    turtlebot4_voice_nav/ # segmentation, commands, navigator, speech, dashboard
    maps/                 # map gốc + lưới nhãn + regions.json + ảnh phân vùng
    config/               # hạt giống và các nhãn chức năng
    test/                 # kiểm thử logic, action và decoder
    tools/                # chuẩn bị sim, kiểm chứng, tạo demo/báo cáo
    artifacts/            # log, WAV, video, thống kê
    docs/                 # PDF tóm tắt và Word chi tiết
```

Package turtlebot4_slam_nav là phần hỗ trợ được dùng lại từ bài trước, cung cấp launcher và cấu hình mô phỏng. Chức năng mới của Bài 7 nằm trong turtlebot4_voice_nav. Khi đóng gói cần giữ cả hai package; chỉ chép node nhận giọng nói mà bỏ launcher/map/config sẽ không tạo được một workspace có thể tái hiện.

Repository nộp không chứa venv, weights ASR, build/install/log hoặc dữ liệu tài khoản. Model được tải qua công cụ riêng và lưu ở cache người dùng. Kiểm tra build trên workspace tách biệt được dùng để giảm nguy cơ vô tình phụ thuộc các package ngoài bộ nộp.

3. KIẾN TRÚC TỔNG THỂ VÀ LUỒNG DỮ LIỆU



Hình 2. Các lớp xử lý và liên kết tới Nav2/Gazebo.

3.1. Phân tách xử lý offline và online

Ở bước offline, chương trình đọc map.yaml, map.pgm và cấu hình hạt giống; tạo region_labels.npy, ảnh tô màu và regions.json. Đây là dữ liệu có thể xem xét trước khi robot chạy. Các tọa độ đích được kiểm tra ngay trong bước tạo dữ liệu và kiểm tra lại lúc node điều hướng khởi động.

Ở bước online, microphone hoặc ô nhập văn bản tạo một thông điệp lệnh. Parser biến thông điệp thành một Command chứa kind, route và reason. Node điều hướng kiểm tra tên vùng, mức tin cậy nếu nguồn là microphone, trạng thái robot và readiness của Nav2. Chỉ khi các kiểm tra đạt, action client mới gửi goal đầu tiên.

Trong lúc robot đi, feedback cập nhật khoảng cách còn lại; chuỗi TF cung cấp vị trí trong bản đồ; /plan cung cấp đường kế hoạch cho giao diện. Khi action trả kết quả, ứng dụng hậu kiểm vị trí trước khi tăng chỉ số chặng. Các sự kiện có ý nghĩa được ghi JSONL với thời gian thực và thời gian mô phỏng.

3.2. Tại sao tách dashboard khỏi bộ điều hướng?

Dashboard là công cụ nhập lệnh và quan sát. Nó không tự tính quỹ đạo để vẽ robot chạy thay cho simulator. Tách giao diện và node action cho phép kiểm thử parser/action không cần màn hình, đồng thời hỗ trợ gửi câu lệnh từ công cụ ROS khác. Nếu dashboard không nhận heartbeat, người dùng thấy trạng thái mất kết nối thay vì một hoạt cảnh tiếp tục chạy.

Giọng nói và lệnh TEXT được ghi khác nguồn. Âm thanh tổng hợp dùng trong kiểm thử cũng có nhãn SYNTHETIC_AUDIO riêng. Việc phân biệt nguồn giúp đánh giá đúng phạm vi bằng chứng: một nút câu mẫu hoạt động không chứng minh microphone đã nhận đúng câu của người nói.

4. BẢN ĐỒ OCCUPANCY GRID VÀ BIẾN ĐỔI TỌA ĐỘ

4.1. Ý nghĩa dữ liệu bản đồ

Ảnh PGM lưu mức xám của từng ô, trong khi file YAML lưu độ phân giải, origin, chế độ và hai ngưỡng diễn giải occupancy. Ảnh là ma trận hàng–cột với gốc ở góc trên trái; frame map của ROS dùng đơn vị mét và hướng trục riêng. Vì vậy vị trí nhìn thấy trên ảnh không thể đưa trực tiếp vào PoseStamped.

Với negate=0, occupancy p=1−I/255, trong đó I là giá trị xám. Ô được xem là free nếu p<free_thresh và occupied nếu p>occupied_thresh; khoảng giữa là unknown. Chương trình giữ unknown ngoài vùng có thể gán đích, kể cả khi nó không phải một pixel đen của tường.

```
image: map.pgm
mode: trinary
resolution: 0.05
origin: [-10, -10, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196
```

4.2. Công thức từ tâm pixel sang tọa độ map

```
x_local = (column + 0.5) × resolution
y_local = (height - row - 0.5) × resolution
x_map = origin_x + cos(yaw)×x_local - sin(yaw)×y_local
y_map = origin_y + sin(yaw)×x_local + cos(yaw)×y_local
```

Hệ số 0,5 chọn tâm ô thay vì góc ô. Thành phần height−row−0,5 đảo chiều hàng ảnh để phù hợp trục y. Khi origin yaw khác 0, bỏ bước quay sẽ gây sai số tăng theo khoảng cách tới gốc. Phép nghịch đảo đưa điểm map về hệ local, chia resolution và floor để tìm ô chứa điểm.

Trên bản đồ 401×401 ô, vùng ảnh có kích thước 20,05×20,05 m. Goal A ở (−2,−3) thuộc pixel (160,260), goal B thuộc (240,260), goal C thuộc (160,120). Tọa độ goal lưu trong JSON là tọa độ được khảo sát; tâm ô chứa A là (−1,975; −2,975), nên không nên nhầm “goal nằm trong ô” với “goal chính xác bằng tâm ô”.

Bảng 4. Những kiểm tra giúp tránh sai hệ tọa độ.

Kiểm tra tọa độ	Mục đích
Round-trip pixel → world → pixel	Trả về đúng ô, kể cả origin có góc quay.
Điểm ngoài ảnh / NaN	Không được xem là goal hợp lệ.
Điểm trên occupied/unknown	Bị loại trước khi gửi Nav2.
Ảnh map và metadata semantic	Đối chiếu SHA-256 để tránh dùng nhầm ảnh bản đồ.

5. PHƯƠNG PHÁP PHÂN VÙNG BÁN TỰ ĐỘNG

5.1. Lựa chọn phương pháp geodesic Voronoi

Mê cung có nhiều lối nối và vùng trống liên thông. Chỉ dùng connected components trên free mask có thể cho một vùng rất lớn thay vì các khu chức năng. Mặt khác, mục tiêu của bài cho phép bắt đầu từ vùng được vẽ/định nghĩa thủ công. Do đó giải pháp chọn hạt giống chức năng và dùng khoảng cách đường đi để chia các ô còn lại.

Mỗi hạt giống có ID và tên. Thuật toán lan đồng thời từ tất cả hạt giống trên lưới free với lân cận bốn hướng. Ô nhận nhãn của hạt giống tới nó bằng số bước ít nhất. Khoảng cách này chịu ảnh hưởng cấu trúc lối đi: hai điểm gần nhau theo đường thẳng nhưng cách bởi tường có thể thuộc các nhánh khác nhau.

Triển khai dùng priority queue lưu (distance, label, row, column). Ban đầu nhãn seed bằng chỉ số vùng, distance=0; các ô khác có distance vô cùng và label=0. Khi lấy một ô ra, thuật toán xét bốn ô kề còn trong ảnh và thuộc free mask, cập nhật nếu có đường ngắn hơn. Chỉ cập nhật khi distance mới nhỏ hơn giúp kết quả ổn định với thứ tự xử lý đã xác định.

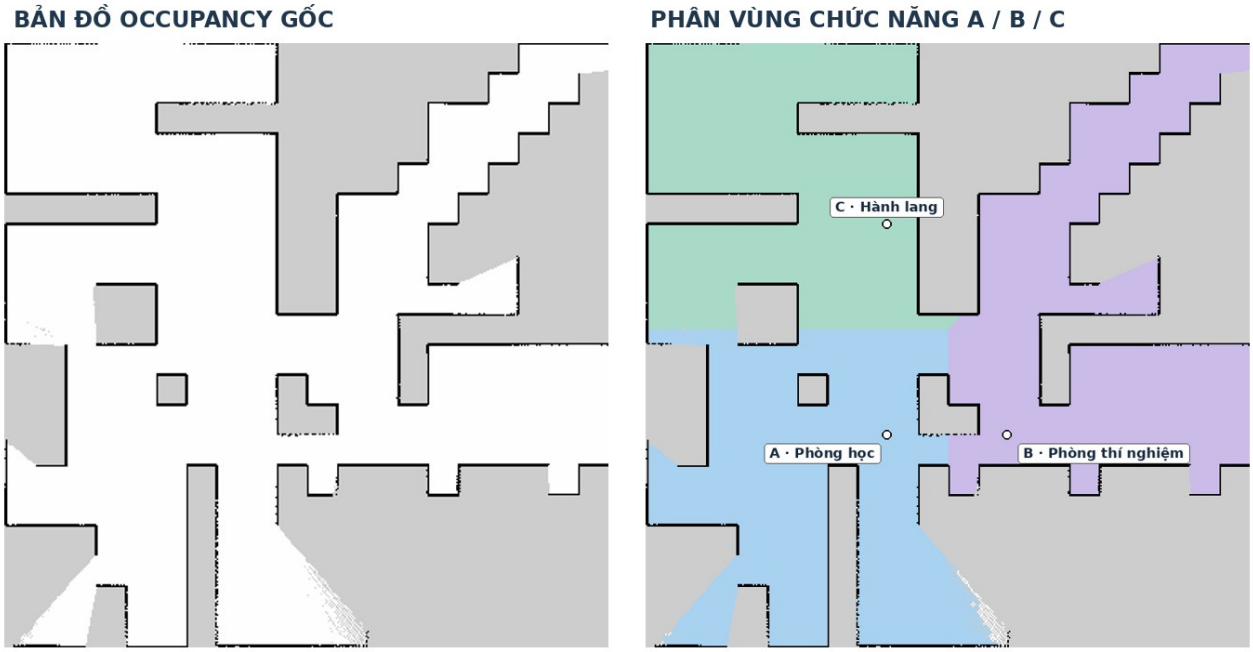
```
for each semantic seed i:
    check seed is known-free and has sufficient clearance
    label[seed_i] = i; distance[seed_i] = 0
    push seed_i into priority queue
while queue is not empty:
    pop cell with smallest distance
    for neighbor in four-connected known-free cells:
        if distance[current] + 1 < distance[neighbor]:
            update distance and copy semantic label
            push neighbor
keep all unknown, occupied, disconnected cells unlabelled
```

5.2. Tính chất, chi phí và giới hạn

Với V ô được thăm và tối đa bốn cạnh mỗi ô, bản triển khai priority queue có chi phí xấp xỉ O(V log V) và bộ nhớ O(V). Vì trọng số các cạnh đều bằng một, có thể thay bằng multi-source BFS để đạt O(V+E); đó là hướng tối ưu hóa, chưa phải cách đang chạy trong source hiện tại.

Phân vùng này không chứng minh vùng A là một phòng học về mặt kiến trúc hay chức năng thực tế. Tên vùng được gán bởi người dùng; thuật toán chỉ chia miền hình học có thể đi. Nếu thay bản đồ hoặc thay seed, phải tạo lại metadata, kiểm tra đích và xem lại ranh giới. Thành phần free không nối bất kỳ seed nào vẫn có label=0; kết quả hiện tại có 187 ô free chưa gán, tương đương 0,4675 m².

6. GẮN NHÃN, DỮ LIỆU VÙNG VÀ ĐÍCH HỢP LỆ



Hình 3. Bản đồ gốc và bản đồ phân vùng, giữ nguyên tường/unknown.

Bảng 5. Các vùng và goal đã lưu trong regions.json.

ID / Tên	Goal (x, y, yaw)	Ô (c,r)	Diện tích	Clearance
A – Phòng học	(-2.0; -3.0; 0.000)	(160,260)	75.0650 m²	1.051 m
B – Phòng thí nghiệm	(2.0; -3.0; 0.000)	(240,260)	73.2625 m²	0.950 m
C – Hành lang	(-2.0; 4.0; 1.571)	(160,120)	71.9500 m²	1.000 m

Diện tích vùng là số ô mang nhãn nhân resolution². Với resolution 0,05 m, mỗi ô tương ứng 0,0025 m². Tổng diện tích ba vùng khoảng 220,2775 m²; con số này chỉ là diện tích free được gán nhãn trên bản đồ hiện có, không phải diện tích kiến trúc chính xác của từng phòng.

Khoảng trống được tính bằng distance transform trên free mask có padding mép ảnh. Tường, unknown và ô ngoài ảnh đều không được coi là free. Hạt giống và goal cần clearance tối thiểu 0,32 m. Chương trình còn kiểm tra goal nằm đúng label của vùng, nhằm tránh trường hợp tên vùng được gắn với một điểm ở nhánh khác.

6.1. Các trường quan trọng trong metadata

```
{
  "id": "B", "name": "Phòng thí nghiệm",
  "aliases": ["khu vực b", "khu vực bê", "phòng thí nghiệm"],
  "goal": {"x": 2.0, "y": -3.0, "yaw": 0.0},
  "goal_pixel": [240, 260], "clearance_m": 0.95,
  "area_m2": 73.2625,
  "polygons": ["các đỉnh biên vùng trong frame map"]
}
```

Đoạn JSON trên rút gọn phần polygons để dễ đọc; tọa độ biên đầy đủ nằm trong file thực tế. Ảnh bản đồ có mã băm SHA-256 trong metadata. Node từ chối khởi động nếu ảnh không khớp mã băm, tránh sử dụng điểm đích đã kiểm tra trên một ảnh khác.

7. THU ÂM VÀ NHẬN DẠNG GIỌNG NÓI TIẾNG VIỆT

7.1. Luồng xử lý âm thanh

Microphone được đọc bằng arecord qua thiết bị ALSA mặc định. Định dạng PCM là mono, 16 kHz, signed int16. Worker đọc từng block 4.000 byte, tương ứng 2.000 mẫu hay 0,125 giây âm thanh. Luồng thu/chạy ASR tách khỏi vòng cập nhật Tkinter nên giao diện không phải chờ từng lần giải mã mới vẽ được bản đồ.

Vosk small-vn-0.4 xử lý âm thanh offline. Các mẫu câu trong grammar hướng bộ giải mã về miền lệnh điều hướng, nhưng grammar vẫn giữ [unk] và một số tên vùng ngoài danh sách để có thể từ chối. Việc thu âm hằng ngày không gọi Google TTS; Google TTS chỉ tạo các câu mẫu thử nghiệm được cache trước đó.

Kết quả tạm dùng để hiển thị người dùng đang nói gì. Lệnh di chuyển chỉ được gửi từ kết quả cuối câu; nếu gửi ngay từ một partial “đi đến A”, người nói có thể chưa nói xong “rồi đến B”. Các câu Stop/Hủy là trường hợp riêng có thể xử lý sớm. Cửa sổ âm thanh tối đa 8 giây giúp tránh chờ vô hạn khi tiếng nền làm Vosk không tìm được điểm hết câu.

Bảng 6. Cấu hình nhận dạng và ý nghĩa vận hành.

Thông số	Giá trị / cách dùng
Tần số mẫu / kênh / mẫu	16.000 Hz / 1 / int16
Kích thước block	4.000 byte, xấp xỉ 125 ms
Model	vosk-model-small-vn-0.4, lưu cache người dùng
Ngưỡng lệnh microphone	Confidence ≥0,65; hủy không bị xử lý như lệnh di chuyển
Kết quả tạm	Hiển thị transcript, chưa gửi tuyến điều hướng
Kết thúc đoạn	Endpoint ASR hoặc chốt cửa sổ 8 giây
Dữ liệu người dùng	Không tự lưu bản thu mic thành file; có log transcript theo thao tác

7.2. Confidence và chất lượng tín hiệu

Confidence được lấy từ các từ nhận dạng, không phải giá trị do giao diện tự gán. Một câu có confidence cao vẫn có thể sai tên khu vực; ví dụ trong quá trình phát triển đã có hiện tượng tên vùng ngoài danh sách bị ép về âm gần giống của vùng hợp lệ. Vì vậy threshold chỉ là một lớp kiểm tra, không thể thay thế kiểm tra ngữ nghĩa hoặc đánh giá trên giọng người thật.

Nếu thanh mức âm gần tối đa liên tục, cần kiểm tra gain/boost của micro thay vì tiếp tục tăng độ nhạy. Mẫu bị clipping làm mất thông tin âm học và có thể khiến ASR giữ partial hoặc nhận sai. Bài 7 giữ cấu hình mức thu đã được điều chỉnh từ bài trước; người dùng nên thử lại trong môi trường yên tĩnh trước khi quay demo.

8. CHUẨN HÓA VĂN BẢN, ALIAS VÀ TUYẾN NHIỀU ĐÍCH

8.1. Từ transcript tới Command

Parser chuyển văn bản sang chữ thường, xử lý chữ đ, bỏ dấu kết hợp Unicode và chuẩn hóa khoảng trắng. Các alias được chuẩn hóa bằng cùng một hàm để so khớp không phụ thuộc hoa/thường hay dấu. Đối tượng Command mang kind, route có thứ tự và reason phục vụ phản hồi.

Danh mục alias hỗ trợ “khu vực A”, “phòng A”, “phòng học”, cách đọc B là “bê” và C là “xê”. Alias là dữ liệu ánh xạ tên sang ID; nếu cùng một alias chuẩn hóa trỏ đến hai vùng khác nhau, chương trình báo lỗi thay vì chọn ngẫu nhiên. Trong bản hiện tại, parser ưu tiên sự rõ ràng hơn là đoán bằng fuzzy matching.

Bảng 7. Một số mẫu phân tích và cách xử lý.

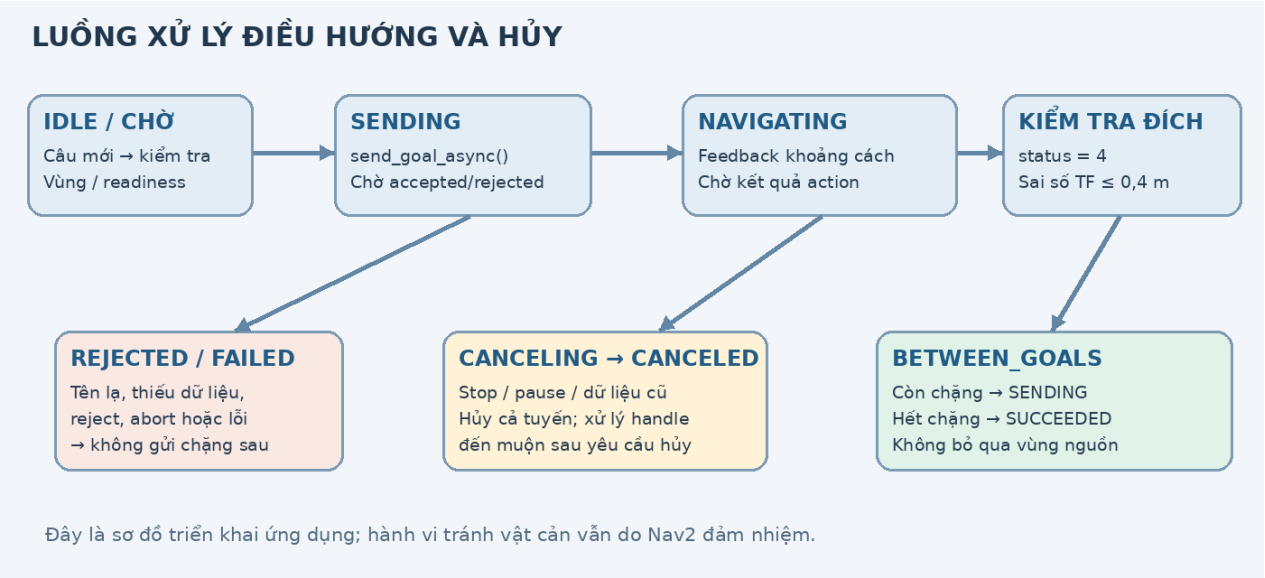
Câu đầu vào	Kết quả phân tích	Ý nghĩa
Đi đến khu vực A.	NAVIGATE → [A]	Một đích có ID.
Đi đến phòng thí nghiệm.	NAVIGATE → [B]	Tên semantic được ánh xạ sang ID.
Di chuyển từ A đến B.	NAVIGATE → [A,B]	Không bỏ chặng nguồn A.
Đi từ phòng A qua hành lang C rồi đến phòng B.	NAVIGATE → [A,C,B]	Tuyến mở rộng nhiều điểm.
Đi từ A qua Z rồi đến B.	UNKNOWN_REGION → []	Từ chối toàn tuyến.
Không đi đến A.	INVALID → []	Không biến câu phủ định thành lệnh đi.
Dừng lại.	CANCEL	Hủy goal đang chạy và phần còn lại.

8.2. Quy tắc không bỏ qua thông tin không hiểu

Sau động từ điều hướng, parser tách tuyến theo các liên từ như từ/đến/qua/rồi đến. Mỗi đoạn phải khớp toàn bộ một alias. “Phòng học ngoài sân” không được cắt còn “phòng học” để tiếp tục chạy; tương tự, một vùng Z ở giữa tuyến không bị bỏ qua chỉ vì A và B ở hai đầu vẫn hợp lệ.

Chuỗi [unk] được giữ trong transcript để parser từ chối. Nếu xóa nhãn này, một câu vốn thiếu từ có thể trở thành lệnh tưởng như đúng. Số chặng tối đa là tám; hai ID trùng nhau liên tiếp được gộp, nhưng thứ tự các vùng khác nhau được giữ. Chương trình chưa hỗ trợ hiểu tất cả cấu trúc tiếng Việt tự do, đại từ mơ hồ hoặc đối thoại hỏi lại nhiều lượt; các trường hợp ngoài grammar cần nói lại bằng câu mẫu.

9. ĐIỀU HƯỚNG BẰNG NAV2 VÀ QUẢN LÝ ACTION



Hình 4. Luồng action và các nhánh hủy/lỗi của node voice_navigator.

9.1. Tạo NavigateToPose goal

Sau khi route được kiểm tra, node lấy x, y, yaw của vùng đầu trong regions.json. PoseStamped có frame_id=map, timestamp theo ROS clock và quaternion từ yaw. Với chuyển động phẳng: qz=sin(yaw/2), qw=cos(yaw/2); qx=qy=0. Không đưa pixel hoặc tọa độ world tùy ý vào action đang mong frame map.

send_goal_async cho phép chờ server nhận goal mà không khóa vòng xử lý. Goal handle được dùng để theo dõi kết quả và gửi cancel. Chương trình phân biệt giai đoạn SENDING với NAVIGATING vì người dùng có thể yêu cầu hủy trước khi response accepted tới. Handle đến muộn vẫn phải được cancel nếu cờ hủy đã bật.

```
goal.pose.header.frame_id = "map"
goal.pose.header.stamp = node.get_clock().now().to_msg()
goal.pose.pose.position.x = target["x"]
goal.pose.pose.position.y = target["y"]
goal.pose.pose.orientation.z = sin(target["yaw"] / 2)
goal.pose.pose.orientation.w = cos(target["yaw"] / 2)
future = action_client.send_goal_async(goal, feedback_callback=... )
```

9.2. Tiêu chí chuyển sang chặng tiếp theo

Kết quả action khác STATUS_SUCCEEDED làm dừng cả tuyến. Với status=4, node đọc TF map→base_link và tính sai số Euclid so với goal. Nếu không đọc được localization hoặc sai số lớn hơn 0,4 m, ứng dụng không tuyên bố hoàn thành. Chỉ khi hợp lệ mới thêm region_id vào completed, tăng index và gửi goal kế tiếp.

Nav2 có bộ goal checker riêng với tolerance vị trí 0,25 m và yaw 0,25 rad trong cấu hình. Kiểm tra ứng dụng bổ sung chỉ đo sai số x–y, không phải một phép chứng nhận độc lập về yaw. Hai tầng kiểm tra có vai trò khác nhau; không nên diễn giải ngưỡng 0,4 m của ứng dụng thành tolerance điều khiển thực tế của Nav2.

10. ĐỊNH VỊ, TRÁNH VẬT CẢN VÀ ĐIỀU KIỆN SẴN SÀNG

10.1. Liên kết map – odom – base_link

AMCL kết hợp bản đồ tĩnh, LiDAR và odometry để cung cấp biến đổi map→odom. Phần odometry/robot cung cấp odom→base_link. Node nhận lệnh chỉ sử dụng pose từ chuỗi TF này trong quá trình điều hướng. Cùng một cặp số x, y nhưng khác frame không có cùng ý nghĩa; đó là lý do cần đặt frame_id=map và kiểm tra TF còn mới.

Đối với world maze vừa khởi động, prepare_sim.py dùng vị trí ground truth đọc từ simulator để đặt initial pose và gọi Undock chính thức. Đoạn chuẩn bị không teleport robot. Việc dùng ground truth chỉ giới hạn ở initial pose của map và world đã biết cùng gốc, cùng phần đối chiếu kết quả. Các goal Nav2 sau đó không dùng ground truth như một bộ điều khiển thay AMCL.

10.2. Các kiểm tra trước và trong hành trình

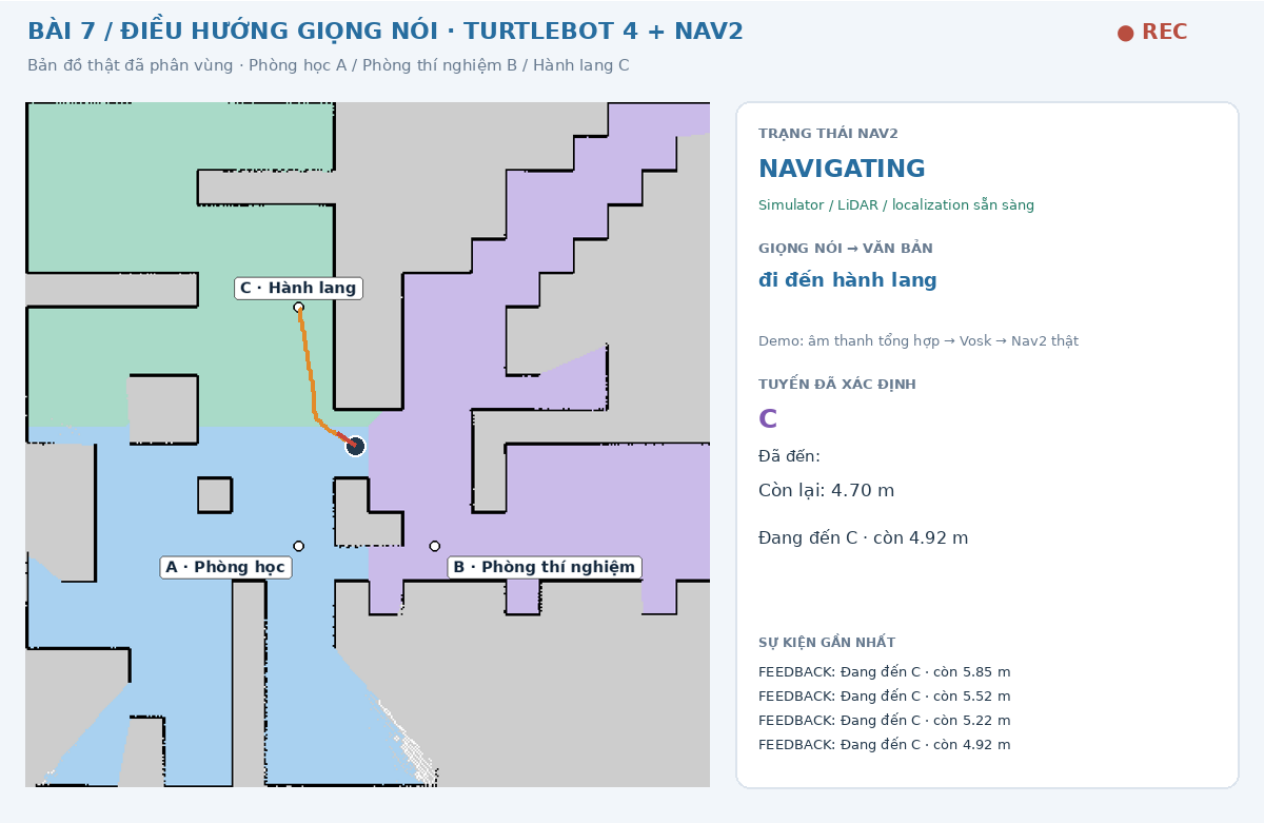
Bảng 8. Các điều kiện vận hành và watchdog.

Điều kiện	Giới hạn đang triển khai	Xử lý khi không đạt
Action server	NavigateToPose phải sẵn sàng	Từ chối trước khi bắt đầu tuyến.
/clock	Có timestamp mới trong 4 s thực	Không bắt đầu / hủy khi đang chạy.
/scan	Có timestamp mới trong 4 s thực	Không tiếp tục khi LiDAR dừng cập nhật.
/dock_status	Mới trong 6 s; is_docked=false	Yêu cầu undock / kiểm tra dữ liệu.
TF map→base_link	Không quá 2 s theo ROS time	Chưa xem localization là hợp lệ.
Goal timeout	240 s theo đồng hồ thực mỗi goal	Yêu cầu hủy và dừng phần còn lại.
Lệnh mới khi bận	Không tự thay route đang chạy	Thông báo bận, cần dừng trước khi đổi.

Watchdog dùng steady clock/monotonic cho thời hạn thực, tránh việc Gazebo pause cũng làm timeout đứng yên. Với /clock và /scan, node kiểm tra timestamp có thay đổi, không chỉ kiểm tra có nhận lại cùng một message. Thông tin cũ được phát lặp không được coi là dữ liệu cảm biến mới.

Tránh vật cản do Nav2 thực hiện thông qua costmap và LiDAR. Cấu hình hiện tại có bán kính robot 0,19 m, inflation_radius 0,45 m, controller_frequency 20 Hz và max_vel_x 0,26 m/s. Các số này là thông số cấu hình, không phải vận tốc đo trung bình của lượt demo. Khi đổi world/map hoặc thêm vật cản động cần kiểm tra lại planner, costmap và khả năng đi qua lối hẹp.

11. GIAO DIỆN, TOPIC ROS VÀ DỮ LIỆU HIỂN THỊ



Hình 5. Ảnh dashboard lấy trong lúc robot thực sự đang đi đến C bằng Nav2.

Bên trái là bản đồ semantic và đường màu cam lấy từ /plan. Ký hiệu robot dựa vào pose TF nhận từ node điều hướng. Bên phải hiển thị trạng thái action, transcript, nguồn âm thanh trong demo, route, các chặng đã đến và khoảng cách feedback. Vì vậy ảnh giao diện vừa giúp thao tác vừa giúp kiểm tra câu lệnh đã được hiểu theo nghĩa nào.

Ảnh này là ảnh chụp trạng thái đang chạy, không phải bằng chứng giọng người thật. Dòng "âm thanh tổng hợp → Vosk → Nav2 thật" được giữ trên giao diện khi kiểm thử bằng WAV. Nút câu mẫu mang nguồn TEXT; không được dùng thay cho một lượt nhận giọng nói trong thống kê ASR.

Bảng 9. Các giao tiếp chính của ứng dụng.

Topic / action	Kiểu dữ liệu	Vai trò
/voice_nav/command	std_msgs/String (JSON hoặc text)	Đầu vào text/source/confidence.
/voice_nav/status	std_msgs/String JSON	Trạng thái, route, completed, pose, lỗi readiness.
/voice_nav/regions	visualization_msgs/MarkerArray	Viền/tên vùng để RViz hiển thị.
/navigate_to_pose	nav2_msgs/action/NavigateToPose	Goal, feedback, result và cancel.
/plan	nav_msgs/Path	Đường kế hoạch từ Nav2, không do dashboard tự tạo.
/scan; /clock; /dock_status	LaserScan; Clock; DockStatus	Dữ liệu kiểm tra hệ thống.
TF map→base_link	tf2 transform	Pose hiện tại để vẽ và hậu kiểm đích.

12. QUY TRÌNH CÀI ĐẶT VÀ CHẠY THỬ

12.1. Chuẩn bị workspace và thư viện

Đặt hai package trong thư mục src của cùng workspace. Source ROS trước khi build; sau build cần source install/setup.bash để tìm package vừa tạo. Môi trường voice sử dụng Python 3.10 hệ thống với system-site-packages. Model Vosk tải một lần, nằm ngoài repository để không làm bộ nộp chứa weights và venv.

```
source /opt/ros/humble/setup.bash
colcon build --symlink-install --packages-select \
  turtlebot4_slam_nav turtlebot4_voice_nav
python3 -m venv --system-site-packages .venv-tb4-voice
.venv-tb4-voice/bin/python -m pip install \
  -r src/turtlebot4_voice_nav/requirements-voice.txt
.venv-tb4-voice/bin/python \
  src/turtlebot4_voice_nav/tools/download_voice_model.py
```

12.2. Ba terminal vận hành

```
# Terminal 1: khởi động maze + robot + localization + Nav2
bash src/turtlebot4_voice_nav/scripts/start_simulation.sh

# Terminal 2: source ROS, workspace và lab_env.bash trước
python3 src/turtlebot4_voice_nav/tools/prepare_sim.py
ros2 run turtlebot4_voice_nav voice_navigator --ros-args \
  -p use_sim_time:=true

# Terminal 3: giao diện và micro tiếng Việt
bash src/turtlebot4_voice_nav/scripts/run_dashboard.sh --micro
```

Trên máy thử nghiệm Intel, overlay Ogre2 và giới hạn camera mô phỏng 5 Hz được dùng để LiDAR hoạt động đúng và giảm tải hiển thị. Các helper tạo overlay trong cache người dùng, không sửa trực tiếp /opt/ros. Mọi terminal cần cùng cấu hình ROS_DOMAIN_ID/RMW và cùng môi trường DDS; terminal khác domain sẽ khiến giao diện không tìm thấy node dù các tiến trình đều đang chạy.

Sau khi khởi động, kiểm tra bản đồ/laser có khớp vị trí robot, action server sẵn sàng, robot đã undock và dashboard không báo lỗi readiness. Không mở thêm teleop hay gửi goal chuột trong lúc đang chạy thử tuyến giọng nói, vì hai nguồn điều khiển đồng thời làm khó xác định nguyên nhân kết quả.

Launcher trong IDE hỗ trợ --bai 7 để mở dashboard và --bai 1/2/3 để mở bài trước. Dashboard không tự khởi động một robot 2D thay thế nếu Nav2 chưa có. Người sử dụng phải giữ simulator và node điều hướng hoạt động, hoặc làm lại các bước chuẩn bị khi vừa khởi động máy.

13. THIẾT KẾ THỬ NGHIỆM VÀ NGUỒN BẰNG CHỨNG

13.1. Phân tầng kiểm chứng

Bảng 10. Phạm vi của từng loại bằng chứng.

Tầng	Đầu vào và hệ thống	Khẳng định được	Không suy ra được
Logic core	Map/alias/câu text; unittest	Tọa độ, đích, tuyến, tên lạ được xử lý theo luật	Độ chính xác giọng người; chất lượng điều khiển.
Decoder	Recognizer giả và WAV cache	UTF-8, giữ partial, chốt câu, [unk]	Tỷ lệ đúng trong môi trường ồn bất kỳ.
ROS action mock	Server NavigateToPose giả, domain 79	Thứ tự, reject, abort, hủy đến muộn	Robot đã chạy ngoài simulator.
Tích hợp Gazebo	WAV tổng hợp → Vosk → Nav2/TurtleBot 4	Tuyến và chuyển động mô phỏng theo action thực	Độ chính xác ASR trên giọng sinh viên.
Giọng người thật	Micro trực tiếp của người dùng	Chưa thu thập bộ đánh giá định lượng	Không báo cáo một tỷ lệ thành công giả.

Trong demo tích hợp, các WAV tiếng Việt được tạo bằng Google TTS từ câu mẫu chung, sau đó cache vào artifacts/asr_google. Bước nhận dạng sử dụng Vosk offline trên dữ liệu PCM thật của WAV. Chương trình chỉ gửi lệnh nếu transcript được giải mã phù hợp route kỳ vọng; không có cơ chế lấy câu dự định nói thay cho kết quả nhận dạng sai.

Thứ tự chạy tích hợp là: đến A; đến phòng thí nghiệm B; từ A đến B; đến hành lang C; thử vùng D. Việc chạy lệnh đến B trước lệnh A→B tạo điều kiện robot đang ở nơi khác A. Điều này giúp kiểm tra đúng yêu cầu phải đến A trước khi quay lại B.

Trước lượt ghi tích hợp đã có một lượt text smoke test đưa robot đến gần A. Vì vậy goal A đầu lượt tích hợp có thể được action chấp nhận gần như ngay lập tức do robot đã nằm trong tolerance. Báo cáo giữ nguyên dữ liệu và giải thích tình huống, không dùng thời gian rất ngắn đó làm bằng chứng robot đi quãng đường dài với tốc độ cao.

13.2. Các file làm căn cứ

integration_results.json lưu câu nói, transcript, confidence, thứ tự goal, pose cuối, telemetry và kết quả toàn lượt. validation.json tổng hợp các điều kiện đã đối chiếu. navigation.jsonl ghi sự kiện node. video_provenance.json ghi nguồn hình/âm thanh và thời điểm đặt các clip âm thanh tổng hợp; video_validation.json ghi số khung đã giải mã. Các log test tách theo nhóm để tránh gộp thử server giả thành thành tích Gazebo.

14. BẢNG KIỂM THỬ TỰ ĐỘNG CHI TIẾT

Bảng 11. 19 phương thức kiểm thử đã đạt theo log.

Nhóm / STT	Nội dung kiểm tra	Kỳ vọng
Core 01	Ba câu lệnh bắt buộc	A; A→B; B đúng.
Core 02	Tuyến mở rộng A qua C đến B	Giữ thứ tự A,C,B.
Core 03	Vùng lạ ở bất cứ vị trí nào	Từ chối toàn tuyến.
Core 04	Phủ định, [unk], thiếu động từ	Không NAVIGATE.
Core 05	Cancel và cách đọc bê/xê	Đúng ý định/alias.
Core 06	Hậu tố lạ sau tên phòng	Không cắt bỏ từ lạ.
Core 07	Pixel/world với origin quay	Round-trip đúng ô.
Core 08	Goal free/clearance/đúng label	Mọi vùng đạt ràng buộc.
Core 09	Vùng trống bị tường ngăn	Không lan qua tường.
Core 10	Unknown, ngoài map, NaN	Goal không hợp lệ.
Decoder 01	Grammar giữ Unicode tiếng Việt	Không chứa token \u giả.
Decoder 02	Partial tuyến đang nói dở	Chưa gửi movement.
Decoder 03	Hết 8 giây; giữ [unk]	Có final; không xóa từ lạ.
Action 01	Đang nơi khác, lệnh A→B	Gửi A rồi B.
Action 02	Tuyến chứa khu vực lạ	Không có goal gửi.
Action 03	Server reject goal đầu	Không gửi chặng sau.
Action 04	Server abort goal đầu	Không gửi chặng sau.
Action 05	Hủy trước response accepted	Cancel handle đến muộn.
Action 06	Mic confidence thấp	Không gửi goal di chuyển.

Các kiểm tra action được thực hiện với server giả trong một ROS domain riêng để không gửi lệnh tới robot đang dùng demo. Server giả là công cụ điều khiển có chủ đích các kết quả khó tái hiện, chẳng hạn reject hoặc acceptance đến muộn. Kết quả này bổ sung cho lượt Gazebo nhưng không chứng minh robot đã tránh một vật cản cụ thể khi xảy ra lỗi đó.

15. KẾT QUẢ ASR VÀ BÀI HỌC TỪ MẪU NHẬN SAI

Bảng 12. Kết quả Vosk trên năm WAV Google TTS đã cache.

Câu nói tổng hợp	Transcript cuối	Confidence	Route / kết quả
Đi đến khu vực A	đi đến khu vực a	0.9976	A
Di chuyển từ khu vực A đến khu vực B	di chuyển từ khu vực a đến khu vực bê	0.9876	A → B
Đi đến phòng thí nghiệm	đi đến phòng thí nghiệm	0.9988	B
Đi đến hành lang	đi đến hành lang	1.0000	C
Đi đến khu vực D	đi đến khu vực đê	0.9972	UNKNOWN_REGION

Bốn câu chỉ định các vùng hợp lệ được giải mã thành đúng tuyến; câu “Đi đến khu vực D” được giải mã tên vùng thành “đê” và parser trả UNKNOWN_REGION. Cách viết transcript không cần giống hoàn toàn câu dự định theo dấu câu/hoa thường; tiêu chí điều hướng là ý định và thứ tự vùng sau chuẩn hóa.

Năm mẫu trên không tạo thành một đánh giá thống kê độ chính xác tiếng Việt. Chúng có cùng nguồn giọng tổng hợp, ít đa dạng về người nói, khoảng cách micro và tiếng nền. Không thể kết luận hệ thống đạt 100% trên giọng người thật chỉ từ năm câu đã chuẩn bị. Báo cáo không tính WER tổng quát vì chưa có tập dữ liệu người nói đa dạng và protocol tương ứng.

15.1. Kết quả eSpeak ban đầu và ý nghĩa

Bảng 13. Các kết quả lỗi ban đầu vẫn được lưu, không dùng thay kết quả cuối.

Câu mẫu eSpeak	Transcript nhận được	Ý định/tuyến thực nhận
đi đến khu vực a	chuyển khu vực bê	INVALID
di chuyển từ khu vực a đến khu vực bê	di chuyển từ khu vực bê đến khu vực xê	B → C
đi đến phòng thí nghiệm	nghiệm chuyển d	INVALID
đi đến hành lang	dừng lại	CANCEL
đi đến khu vực z	nghiệm khu vực xê	INVALID

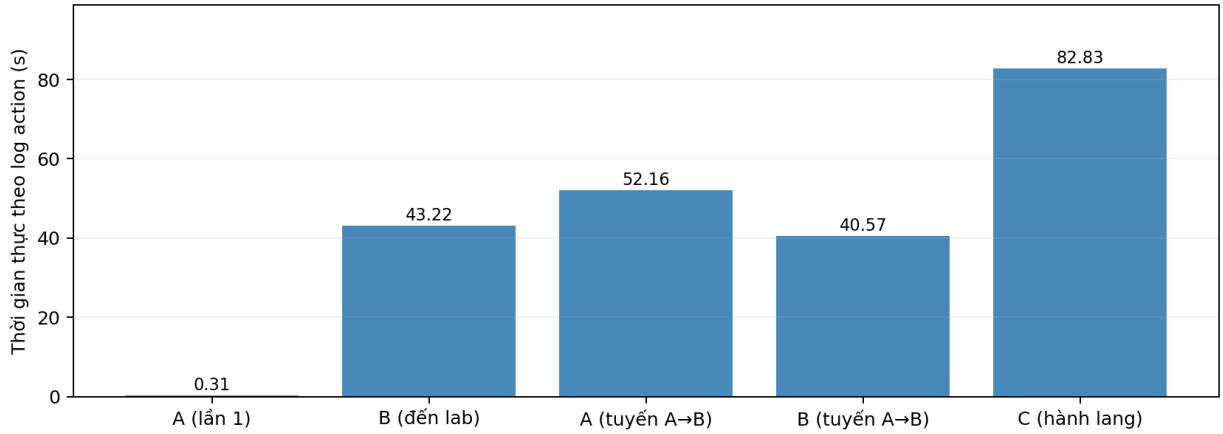
Một số mẫu eSpeak bị nhận thành ý định khác, kể cả trường hợp tuyến dự kiến A→B lại thành B→C với confidence khá cao. Điều này cho thấy giọng tổng hợp có cách phát âm không phù hợp vẫn làm hệ thống sai, và confidence không phải bảo đảm ngữ nghĩa. Ở ca mẫu vùng lạ, việc parser không tạo route chỉ chứng minh không phát lệnh trong mẫu đó, chưa đồng nghĩa đã nhận đúng chữ D.

Ngoài kiểm tra bằng hàm giải mã WAV, công cụ validate_live_decoder.py đưa chính các WAV qua make_recognizer và UtteranceDecoder dùng bởi microphone. Bước này phát hiện khác biệt mã hóa grammar: cần ensure_ascii=False để Vosk nhận từ tiếng Việt UTF-8 thực thay vì chuỗi ký tự \uXXXX. Kiểm thử hồi quy riêng đã được thêm để tránh tái xuất hiện lỗi này.

16. KẾT QUẢ ĐIỀU HƯỚNG GAZEBO THEO TỪNG GOAL

Bảng 14. Năm goal thực Nav2 được kiểm tra trong lượt tích hợp.

Lần	Goal / câu lệnh	Pose cuối TF (x,y)	Sai số (m)	Thời gian thực (s)
1	A – đã ở gần A	(-1.9334; -2.7721)	0.237392	0.314
2	B – đến lab	(1.8111; -3.1345)	0.231854	43.218
3	A – trong tuyến A → B	(-1.8660; -3.1203)	0.180101	52.164
4	B – trong tuyến A → B	(1.8472; -3.1749)	0.232296	40.573
5	C – hành lang	(-1.8998; 3.7860)	0.236293	82.825



Hình 6. Thời gian từ GOAL_SENT đến GOAL_SUCCEEDED theo wall_time; goal A đầu đã ở gần đích.

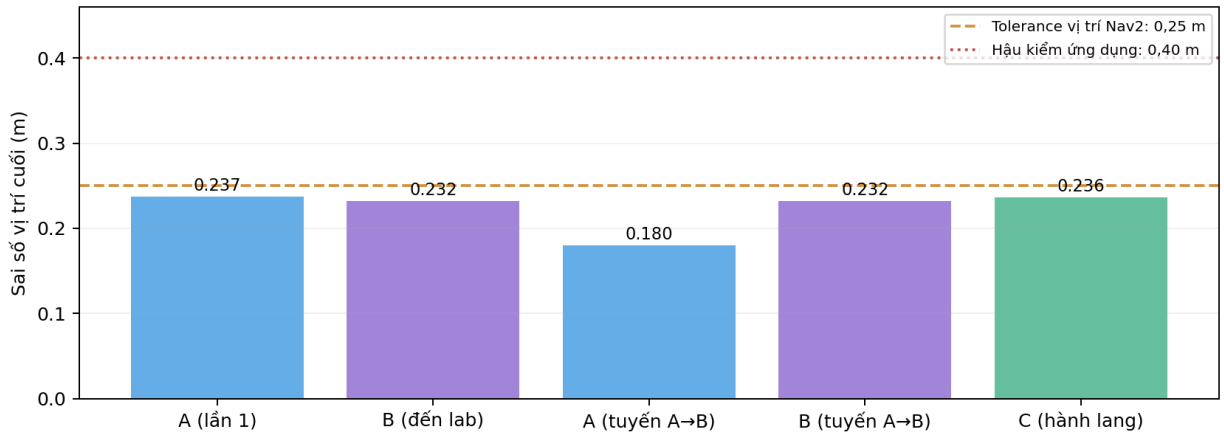
16.1. Diễn giải các chặng

Goal A đầu tiên hoàn thành trong khoảng 0,314 s vì robot đã nằm gần A sau lượt text smoke test trước đó. Pose bắt đầu khoảng (−1,9334; −2,7721), gần goal (−2; −3). Vì vậy không dùng kết quả này để suy ra tốc độ di chuyển hoặc thời gian đi từ gốc bản đồ tới A.

Lệnh đến phòng thí nghiệm đưa robot từ gần A sang B trong khoảng 43,218 s. Tiếp theo, khi robot đang gần B tại (1,7997; −3,1429), lệnh “Di chuyển từ khu vực A đến khu vực B” thực hiện hai goal: đến A mất 52,164 s rồi về B mất 40,573 s. Đây là bằng chứng trực tiếp cho quy tắc không bỏ qua vùng nguồn.

Chặng đến hành lang C bắt đầu gần B và kết thúc gần (−1,8998; 3,7860), mất 82,825 s theo đồng hồ thực. Tổng thời gian action của năm goal khoảng 219,095 s; tổng lượt tích hợp 229,939 s còn bao gồm xử lý lệnh và khoảng dừng giữa các lượt. Không nhầm các thời gian này với thời gian simulation khi simulator chạy chậm hơn thời gian thực.

17. PHÂN TÍCH SAI SỐ, QUỸ ĐẠO VÀ HAZARD



Hình 7. Sai số vị trí cuối từng goal và các ngưỡng cấu hình/hậu kiểm.

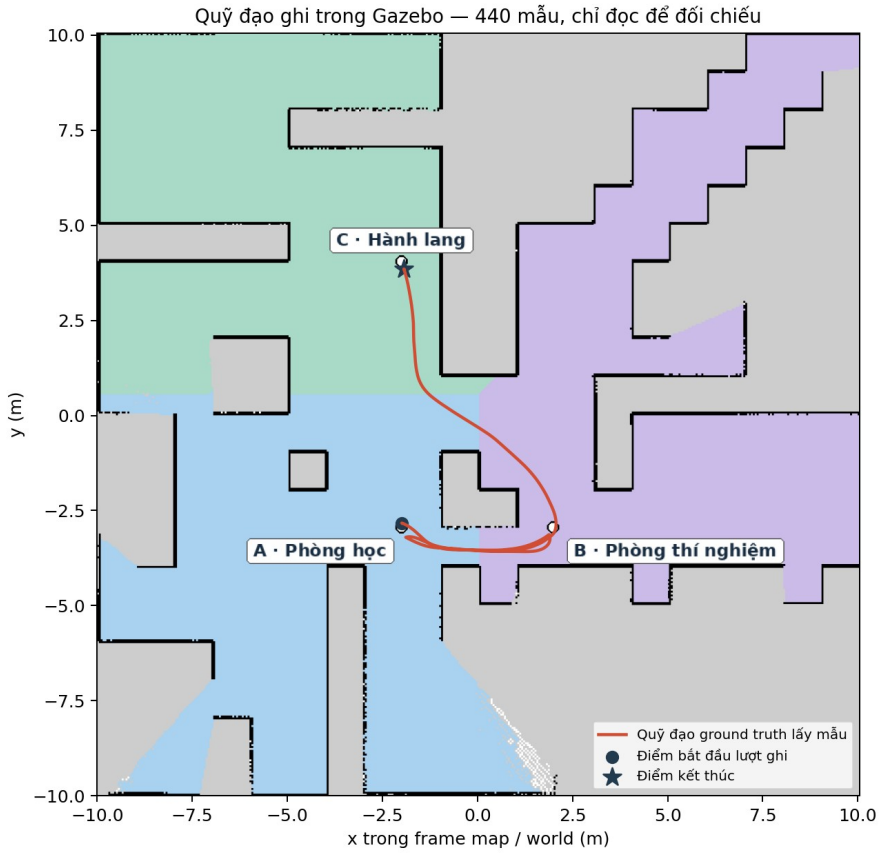
Bảng 15. Thống kê tính lại từ dữ liệu action/TF và đối chiếu Gazebo.

Đại lượng	Giá trị	Cách diễn giải
Sai số nhỏ nhất	0.180101 m	Goal A trong tuyến A → B.
Sai số lớn nhất	0.237392 m	Không vượt 0,25 m trong năm mẫu x–y ghi nhận.
Trung bình	0.223587 m	Trung bình số học của năm sai số Euclid.
Trung vị	0.232296 m	Giá trị giữa khi sắp xếp.
RMSE	0.224653 m	Căn trung bình bình phương các sai số.
Độ lệch chuẩn tổng thể	0.021851 m	Mô tả năm mẫu, không phải ước lượng độ tin cậy của mọi lượt.
Khoảng cách tâm–tường nhỏ nhất	0,419981 m	Theo 440 mẫu ground truth và box vách maze.

Sai số ở đây được tính bằng sqrt((x_TF−x_goal)²+(y_TF−y_goal)²) tại thời điểm đọc sau action. Nó bao gồm ảnh hưởng tolerance của goal checker, sai số localization và thời điểm lấy mẫu; không phải chỉ là sai số cơ khí. Không có đủ dữ liệu để tách riêng từng nguồn sai số hoặc suy ra phân phối xác suất trên nhiều môi trường.

Không ghi nhận sự kiện BUMP, CLIFF, STALL hoặc WHEEL_DROP trong dữ liệu hazard của lượt này. Tuy nhiên, không có hazard không đồng nghĩa một chứng minh toán học không va chạm. Khoảng cách 0,420 m là khoảng cách nhỏ nhất tại các mẫu đã lấy đối với các box vách được xét; chưa đánh giá liên tục mọi thời điểm, mọi mesh hoặc vật cản động chưa xuất hiện trong lượt thử.

18. ĐỐI CHIẾU QUỸ ĐẠO VÀ XỬ LÝ KHU VỰC LẠ



Hình 8. Quỹ đạo ground truth đã lấy mẫu trong lượt tích hợp, chồng lên bản đồ cùng frame đã chuẩn bị.

Đồ thị sử dụng đúng các mẫu lưu trong integration_results.json, không phải đường nội suy được tạo để thay kết quả Nav2. Các lần đi qua A và B có thể chồng lên nhau nên một đường màu đơn không thể hiện riêng thứ tự; thứ tự được đối chiếu thêm bằng chuỗi GOAL_SENT/GOAL_SUCCEEDED trong log.

Sau khi hoàn thành chặng C, câu “Đi đến khu vực D” được Vosk giải mã thành “đi đến khu vực đê”. Vì alias này không thuộc A/B/C, parser trả UNKNOWN_REGION. Kiểm thử kiểm tra có sự kiện REJECTED và không có GOAL_SENT cho yêu cầu đó. Robot không được phép chọn một khu vực gần âm để thay cho vùng không tồn tại.

Ảnh/video của phiên kiểm thử cũ có thể vẫn giữ route C và dòng SUCCEEDED của yêu cầu trước bên cạnh thông báo từ chối mới. Source cuối đã chỉnh giao diện/trạng thái ứng dụng để yêu cầu bị từ chối hiển thị REJECTED và xóa route cũ. Cải tiến hiển thị này không thay đổi dữ liệu chuyển động hoặc biến yêu cầu D thành một goal hợp lệ.

19. LỖI ĐÃ GẶP, CÁCH KHẮC PHỤC VÀ KIỂM SOÁT CHẤT LƯỢNG

Bảng 16. Các vấn đề kỹ thuật và biện pháp đã áp dụng.

Vấn đề	Nguyên nhân / rủi ro	Cách xử lý trong bản hiện tại
Trùng tên self.handle	rclpy.Node có thuộc tính handle nội bộ	Đổi thành goal_handle, giữ riêng action handle.
Tiếng Việt biến thành \uXXXX	Grammar JSON dùng escape Unicode khi Vosk cần từ UTF-8	Dùng ensure_ascii=False; test grammar và pipeline runtime.
Partial bị giữ lâu	Tiếng nền làm thiếu endpoint	Hiển thị partial; chốt cửa sổ tối đa 8 s.
Partial mất phần đích sau	Gửi câu khi người dùng chưa nói hết	Không phát movement từ partial ổn định.
Vùng D nhận nhầm B	Grammar nhỏ thiên về tên hợp lệ gần âm	Bổ sung cách đọc đê/dê và giữ từ lạ; kiểm tra semantic.
Nhận lại dữ liệu cũ	Có message nhưng timestamp không tiến	Theo dõi clock/scan có timestamp mới.
Hủy trước accepted	Goal handle chưa tồn tại khi người dùng Stop	Ghi cờ hủy; cancel ngay khi handle đến.
set -u khi source ROS	Script setup đọc biến tùy chọn chưa đặt	Launcher không bật nounset trước source.
Tải hiển thị mô phỏng	Cạnh tranh tài nguyên GPU/CPU	Overlay Ogre2 và camera 5 Hz trên máy thử.

Các lỗi được xử lý theo chuỗi tái hiện → sửa ở lớp liên quan → chạy kiểm tra phù hợp. Ví dụ, lỗi UTF-8 không thể được phát hiện chỉ bằng test parser trên chuỗi Python đã đúng; cần đưa WAV qua đúng hàm tạo recognizer mà microphone sử dụng. Tương tự, một server action giả rất hữu ích để tái hiện acceptance đến muộn, nhưng phải giữ nó trong domain riêng để không làm nhiễu Nav2 đang hoạt động.

Không xóa dấu vết kết quả ASR eSpeak sai để trình bày một tỷ lệ hoàn hảo. Báo cáo chỉ dùng các log thực tế để tính số liệu, ghi rõ loại đầu vào và các thay đổi sau lượt quay. Điều này giúp người đọc kiểm tra được lời khẳng định nào dựa trên test logic, lời khẳng định nào dựa trên robot mô phỏng và phần nào chưa được đo.

20. GIỚI HẠN, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

20.1. Giới hạn hiện tại

Phân vùng dựa vào hạt giống chức năng nên cần người dùng hiểu bản đồ để đặt seed và tên phù hợp. Ranh giới geodesic không nhất thiết trùng cửa/phòng vật lý. Nếu muốn tự nhận phòng, cần bổ sung tiêu chí hình thái, phát hiện cửa hẹp, watershed hoặc mô hình phân loại không gian và đánh giá bằng dữ liệu gán nhãn độc lập.

Bộ parser hỗ trợ một miền câu nhỏ và so khớp alias chặt. Cách này giảm suy đoán sai nhưng làm các câu diễn đạt khác mẫu dễ bị từ chối. ASR cũng chưa được đánh giá trên nhiều giọng, tốc độ nói, khoảng cách micro và tiếng ồn. Điểm tin cậy cao không loại trừ việc nhận sai một chữ cái; ứng dụng thực tế cần xem xét bước xác nhận cho các lệnh ảnh hưởng lớn.

Thử nghiệm mới dùng một world maze với một bản đồ đã lưu và năm goal của một lượt tích hợp. Không có cơ sở để suy rộng tỷ lệ thành công sang mọi bản đồ, mọi mật độ vật cản hoặc robot vật lý. Độ an toàn khi gặp người đi ngang, vật cản động và thay đổi môi trường còn cần các kịch bản riêng.

20.2. Các mở rộng đề xuất

- Cho phép người dùng chọn/vẽ vùng trực tiếp trên bản đồ, xem khoảng trống của goal và lưu phiên bản seed/metadata trước khi chạy.
- Tự phát hiện lối hẹp và vùng phòng bằng morphology/watershed; giữ nhãn chức năng là lớp dữ liệu tách khỏi thuật toán hình học.
- Bổ sung bước robot đọc lại tuyến bằng tiếng nói và yêu cầu xác nhận khi nhiều tên vùng gần âm hoặc confidence không chắc chắn.
- Thu bộ dữ liệu người thật có ground-truth transcript để tính WER, đúng ý định, đúng tuyến, false activation và độ trễ từ hết câu đến gửi goal.
- Thử nhiều điểm xuất phát và vật cản động; đánh giá khoảng cách tối thiểu, số recovery, độ dài đường và thời gian theo simulation lẫn wall clock.
- Lưu version bản đồ, YAML transform và config cùng mã băm; tái kiểm tra đích tự động khi có thay đổi dữ liệu hoặc phần mềm.

20.3. Kết luận kỹ thuật

Hệ thống đã nối được bản đồ có nhãn với lệnh tiếng Việt và thực thi route tuần tự bằng Nav2 trong TurtleBot 4 Simulation. Kết quả đáng chú ý là tuyến A→B vẫn đi qua A khi robot đang ở B, vùng không tồn tại không tạo goal, và các điều kiện lỗi có đường hủy/dừng rõ ràng. Các kết quả được lưu bằng log và video để có thể đối chiếu; phần giọng người thật và công bố repository public vẫn là hạng mục cần hoàn tất khi có dữ liệu/kết nối tương ứng.

21. HƯỚNG DẪN QUAY DEMO VÀ DANH MỤC NỘP BÀI

21.1. Kịch bản quay lại bằng giọng người thật

Bảng 17. Kịch bản bổ sung bằng người nói thật.

Bước	Thao tác	Thông tin cần thấy trên video
1	Mở Gazebo, RViz/dashboard; kiểm tra robot undock	Map đúng world, robot và LiDAR/localization sẵn sàng.
2	Bật micro và nói Đi đến khu vực A	Giọng người, transcript, route A và quá trình robot tới đích.
3	Nói Đi đến phòng thí nghiệm	Alias tên semantic ánh xạ B và robot di chuyển.
4	Khi đang B, nói Di chuyển từ A đến B	Robot đi A trước, thông báo đã đến A, sau đó B.
5	Nói Đi đến hành lang	Route C và pose cuối gần goal C.
6	Nói Đi đến khu vực D	Transcript vùng lạ, REJECTED, không có chuyển động mới.
7	Có thể thử Stop giữa đường	Thấy yêu cầu cancel và robot dừng; gửi lại lệnh mới để chạy.

Nút ghi video nội bộ ghi hình dashboard và câu ASR nhưng không thu bản âm thanh microphone của người dùng. Nếu cần đáp ứng video thể hiện rõ người ra lệnh bằng giọng nói, dùng trình quay màn hình có bật micro và đưa Gazebo/RViz cùng dashboard vào khung. Nên bắt đầu quay trước khi phát câu lệnh, giữ nguyên thứ tự sự kiện và không cắt một đoạn nhận sai thành đoạn nhận đúng.

Video hiện có DEMO_BAI07.mp4 dài 264,2 s dùng hình điều hướng thật và các clip Google TTS đã cache để nghe lại đầu vào thử nghiệm. Thời gian chuyển động được giữ nguyên, không tăng tốc để làm robot có vẻ hoàn thành nhanh hơn. video_provenance.json ghi lịch đặt audio; dòng chữ trên video nêu rõ nguồn âm thanh tổng hợp.

21.2. Nội dung nộp và trạng thái công bố

- Source: hai package ROS trong repository local đã commit; kèm README và lệnh cài đặt/build để tái hiện.
- Bản đồ: map.pgm, map.yaml, segmented_map.png, region_labels.npy, regions.json và region_seeds.yaml.
- Bằng chứng: video MP4, WAV mẫu, log điều hướng, số liệu kiểm chứng và log 19 test.
- Báo cáo: PDF tóm tắt 2 trang đúng đề, bản Word chi tiết bổ sung để chỉnh sửa/thuyết trình.

Ở thời điểm lập báo cáo, chưa xác nhận có repository GitHub public đã push thành công. Commit local không phải đường dẫn GitHub. Khi có kết nối GitHub cần tạo/push repository public, kiểm tra truy cập từ bên ngoài và cập nhật URL vào trang bìa hoặc phụ lục; không điền một URL giả để hoàn thiện hình thức.

22. PHỤ LỤC: LỆNH TÁI HIỆN, TRÍCH LOG VÀ TÀI LIỆU

22.1. Lệnh kiểm tra và truy vết

```
source /opt/ros/humble/setup.bash
source install/setup.bash
export PYTHONPATH="$PWD/src/turtlebot4_voice_nav:$PYTHONPATH"
python3 -m unittest discover -s src/turtlebot4_voice_nav/test \
  -p test_core.py -v
python3 -m unittest discover -s src/turtlebot4_voice_nav/test \
  -p test_speech_decoder.py -v
ROS_DOMAIN_ID=79 ROS_LOCALHOST_ONLY=1 \
  RMW_IMPLEMENTATION=rmw_fastrtps_cpp \
  python3 src/turtlebot4_voice_nav/test/test_action_flow.py
.venv-tb4-voice/bin/python \
  src/turtlebot4_voice_nav/tools/validate_live_decoder.py
python3 src/turtlebot4_voice_nav/tools/validate_evidence.py
```

22.2. Mẫu cấu trúc sự kiện log

```
{"event":"GOAL_SUCCEEDED", "command":"di chuyển từ khu vực a đến khu vực bê",
 "source":"SYNTHETIC_AUDIO", "route":["A","B"], "region_id":"A",
 "pose":[-1.865997,-3.120332,0.265405], "error_m":0.180101,
 "action_status":4}

{"event":"REJECTED", "command":"đi đến khu vực đê",
 "message":"Không có khu vực ...", "source":"SYNTHETIC_AUDIO"}
```

Các ví dụ trên được rút gọn trường và số chữ số để trình bày; file JSON/JSONL gốc là nguồn đối chiếu đầy đủ, không dùng đoạn rút gọn này để tính lại thống kê.

22.3. Tài liệu tham khảo và dữ liệu gốc

[1] TurtleBot 4 User Manual – Simulator: https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_simulator.html

[2] Nav2 Humble – định nghĩa action trên máy: /opt/ros/humble/share/nav2_msgs/action/NavigateToPose.action.

[3] Vosk Models – model small-vn-0.4: https://alphacephei.com/vosk/models.

[4] gTTS – tạo câu mẫu tổng hợp: https://gtts.readthedocs.io/en/latest/module.html.

[5] Mã nguồn thực nghiệm: segmentation.py, commands.py, navigator.py, speech.py, dashboard.py; cấu hình nav2.yaml và region_seeds.yaml.

[6] Dữ liệu đối chiếu: integration_results.json, validation.json, live_decoder_validation.json, asr_google/asr_results.json và ba file log test.

[7] Nguồn video: DEMO_BAI07.mp4, video_provenance.json và video_validation.json; ngày thử 07/11/2026.